

From Experimenter to Architect

Moving from Google Colab to GPUs & Virtual Machines — **Deep Learning**
Indaba GPU Workshop, Nigeria 2026

SHERIFF

OPE

ALEXANDER



Before We Begin

What You Need to Walk In With



Python + ML Fluency

Comfortable with PyTorch or TensorFlow training loops and ML experimentation workflows.



A Laptop or Phone

Plus familiarity with Jupyter Notebook or Google Colab — you've shipped experiments before.



Basic Terminal Comfort

Know your way around `cd`, `ls`, and `ssh`. That's all we ask.

Life in Colab: Why Everyone Starts There



Instant GPU Access

Free GPU in the browser — no credit card, no setup, no DevOps required.



Great for Learning

Perfect for tutorials, prototyping, and getting your first model running fast.

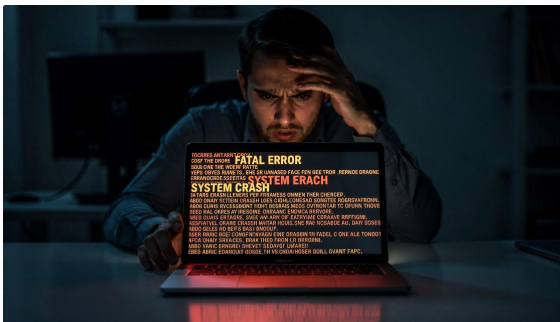


But It Has a Ceiling

Colab is an **experimental lab**, not a production architecture. That distinction matters.

Where Colab Breaks Down

Promising ideas die at "it keeps crashing." This is the ceiling we're here to break through.



⌚ Runtime Timeouts

Long training jobs get killed without warning.

💥 Kernel Crashes

Memory pressure causes crashes as workloads grow.

💾 No Persistent Storage

Serious pipelines need a real filesystem — Colab doesn't provide one.

📊 Shared, Unpredictable GPUs

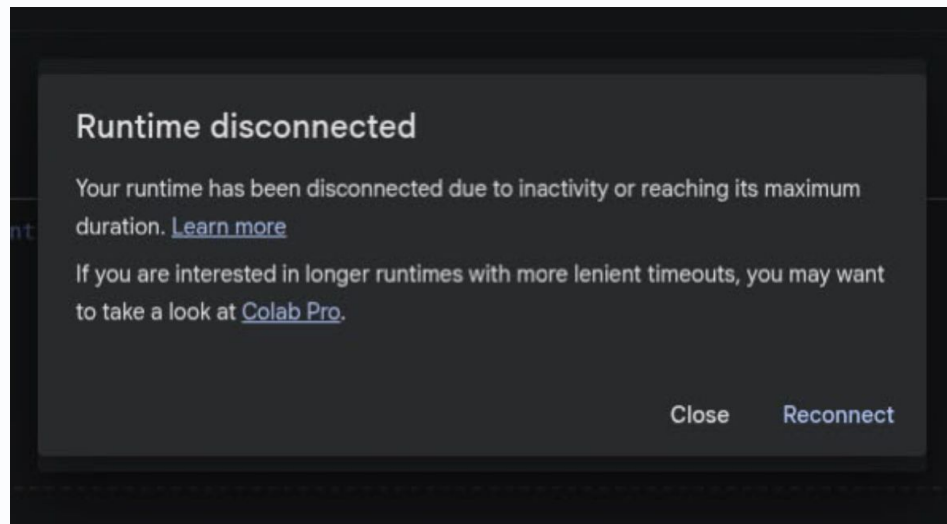
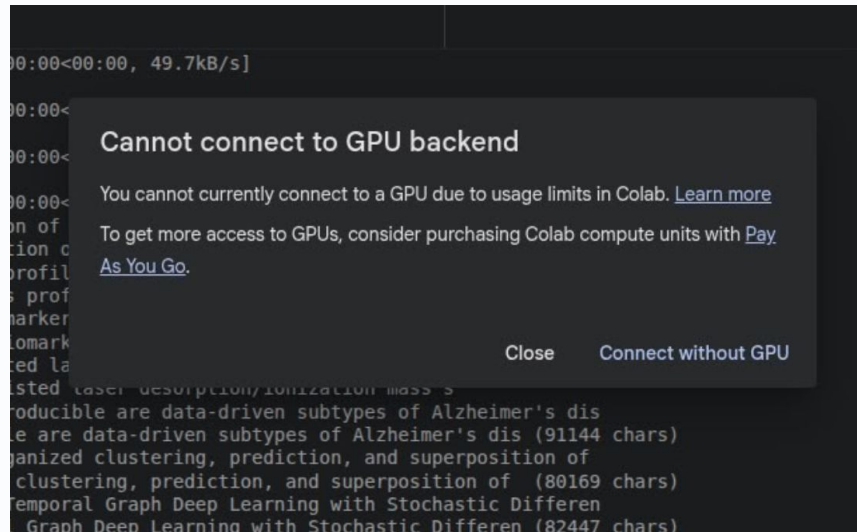
You don't own the hardware — performance varies wildly.

🚫 Credit & Quota Limits

Scale up and the free tier vanishes fast.

⊗ Promising ideas die at "it keeps crashing." This is the ceiling we're here to break through.

Example Failures





Demo Use Case: RAG on a "Small" Model

We'll look to build a **Retrieval-Augmented Generation (RAG)** system using a small open LLM to keep costs low. On Colab, the notebook became unresponsive and hit resource limits mid-experiment.

1

Recreate It

We'll rebuild a thin version of that RAG notebook together.

2

Hit the Wall

See exactly why and where Colab struggles on this workload.

3

Escape the Ceiling

Move compute to a real GPU and watch it fly.

Escaping the Ceiling: GPU Virtual Machines



Think of a VM as a Fresh Computer in the Cloud

You choose your hardware: CPU, RAM, GPU, and disk. You install exactly what you need — Python, CUDA, PyTorch, your libraries.

Key Differences from Colab

- **Persistent** environment and storage
- **You control** runtime, versions, and tools
- Run long jobs **reliably**, overnight, unattended

Keeping Familiar UX: Colab → Remote GPU



Run Jupyter

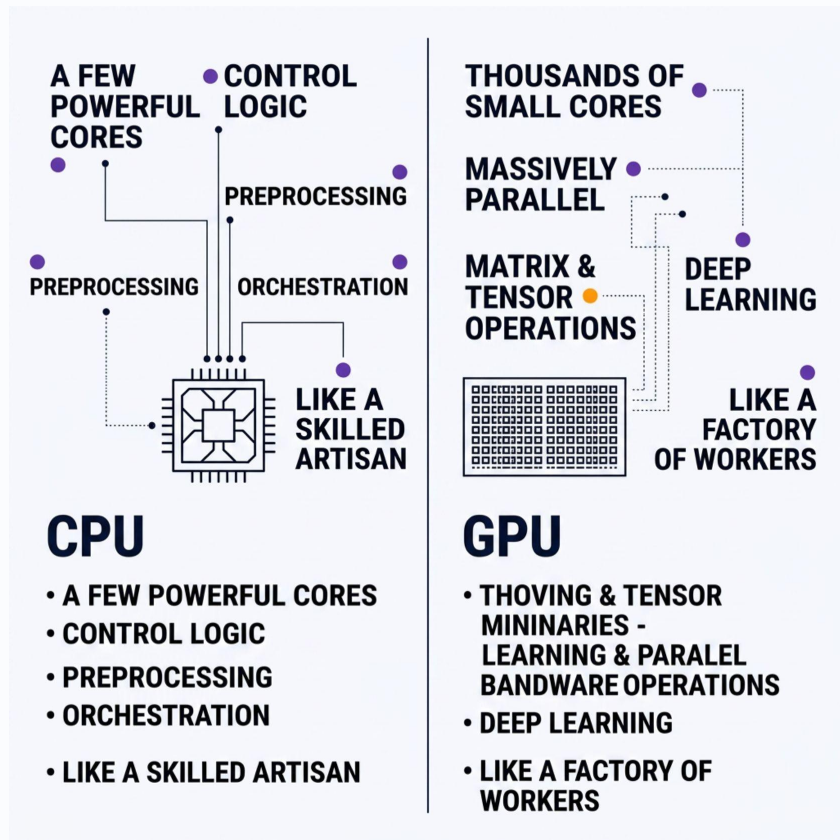
Expose URL

Connect Colab

Use Real GPU

You keep the Colab notebook interface you already know — but swap out Google's shared GPU for your own cloud or local GPU. Ope will demo this live: the same RAG notebook running on a local GPU and then on a remote VM.

CPUs vs GPUs – The "Why" Before We Scale



Why Does This Matter?

Before we talk multi-GPU, we need to understand **why GPUs exist at all**. Deep learning is mostly big matrix math – and GPUs were purpose-built to do exactly that in parallel.

A CPU is a **skilled artisan** handling complex sequential tasks. A GPU is a **factory floor** – thousands of workers doing simple operations simultaneously.

When CPU Is Enough vs When You Need a GPU

✓ CPU Is Enough

- Small classical ML models
- Light inference workloads
- Preprocessing and orchestration
- Debugging logic and data pipelines

⚡ You Need a GPU When...

- Training deep neural networks on large datasets
- Fine-tuning LLMs or vision models
- Serving high-throughput or low-latency inference
- Working with embeddings at scale

This is why Colab's "free GPU" feels magical — and why you very quickly want *more* of it.

Parallelism: Know the Term

When the Model Doesn't Fit on One GPU

Some models — particularly very large LLMs — are simply too big to load into a single GPU's memory. The solution: **split the model itself** across multiple GPUs, with different layers or blocks on different devices.

This is significantly more complex to implement than data parallelism. For most of your work, **data parallelism will get you further, faster.**

Data Parallelism

Model is copied; **data is split**. Your go-to scaling strategy.

Model Parallelism

Model is split across GPUs. Complex — used for very large models.

Accelerate in Practice: Code Comparison

Single GPU (PyTorch Native)

```
import torch

# 1. Setup device
device = "cuda" if torch.cuda.is_available() else "cpu"

# 2. Define standard components
model = MyModel().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
train_dataloader = DataLoader(dataset, batch_size=32)

# 3. Training Loop
for batch in train_dataloader:
    # Manually move data to the device
    inputs, targets = batch[0].to(device), batch[1].to(device)

    outputs = model(inputs)
    loss = loss_fn(outputs, targets)

    # Standard PyTorch backward pass
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

Multi-GPU (with Accelerate)

```
import torch
# NEW: Import the accelerator library
from accelerate import Accelerator

# NEW: Initialize the accelerator object
accelerator = Accelerator()

# 2. Define standard components (No .to(device)!)
model = MyModel()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
train_dataloader = DataLoader(dataset, batch_size=32)

# NEW: Pass components to preparation function
model, optimizer, train_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader
)

# 3. Training Loop
for batch in train_dataloader:
    # NEW: No manual .to(device) for inputs/targets!
    inputs, targets = batch[0], batch[1]

    outputs = model(inputs)
    loss = loss_fn(outputs, targets)

    # NEW: Replace loss.backward() with accelerator wrapper
    accelerator.backward(loss)

    optimizer.step()
    optimizer.zero_grad()
```

Key Differences

Automated Data Distribution

`accelerator.prepare()` automatically configures your `DataLoader` to distribute batches across all available GPUs, managing device placement without explicit `.to(device)` calls.

Synchronized Gradient Updates

`accelerator.backward(loss)` replaces the native PyTorch call, ensuring gradients are computed locally on each GPU and then efficiently averaged across the entire cluster.

Key Takeaways from the Workshop

For more, Connect with team@udutech.co



Start Small, Scale Smart

Begin with accessible tools like Colab/Jupyter, then migrate to dedicated GPU VMs as your models and data grow.



GPUs for Parallelism

Understand that GPUs are purpose-built for the massive parallel matrix computations inherent in deep learning.



Accelerate Your Training

Leverage libraries like Accelerate to easily implement data parallelism and scale across multiple GPUs.



Build Resilient Workflows

Utilize tools like `nvidia-smi` for monitoring and `tmux` for session persistence in any environment.



Support Local Infra

Embrace and contribute to the growth of local and African GPU infrastructure to foster domestic AI talent and ecosystems.